Approach Document

ExpenseAudit — Policy-First Expense Auditor

Name: Adwin T Sunil
Date: April 2026

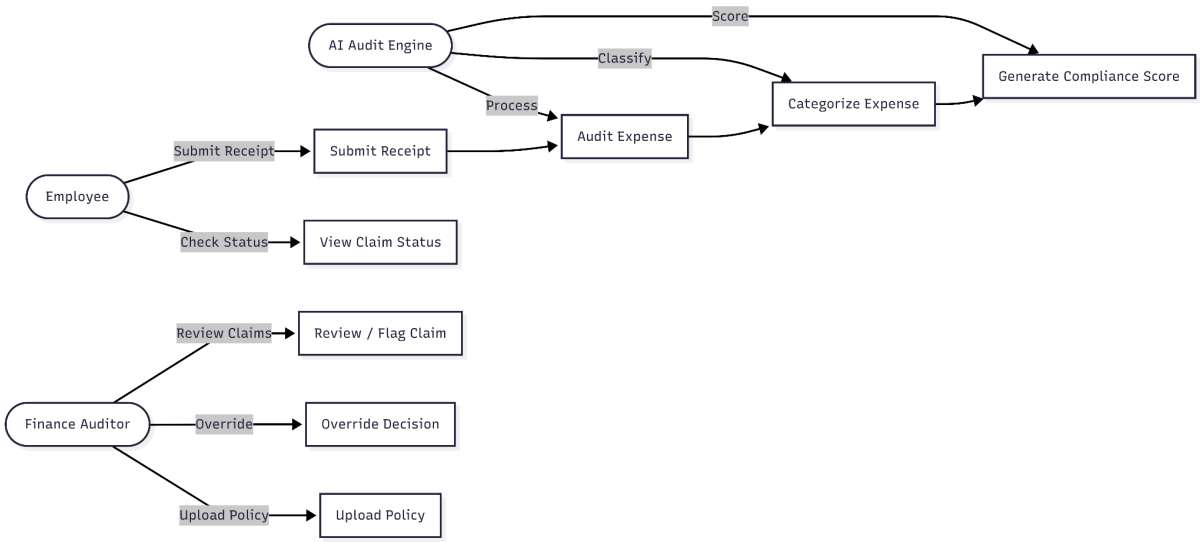
---

1. Problem Understanding

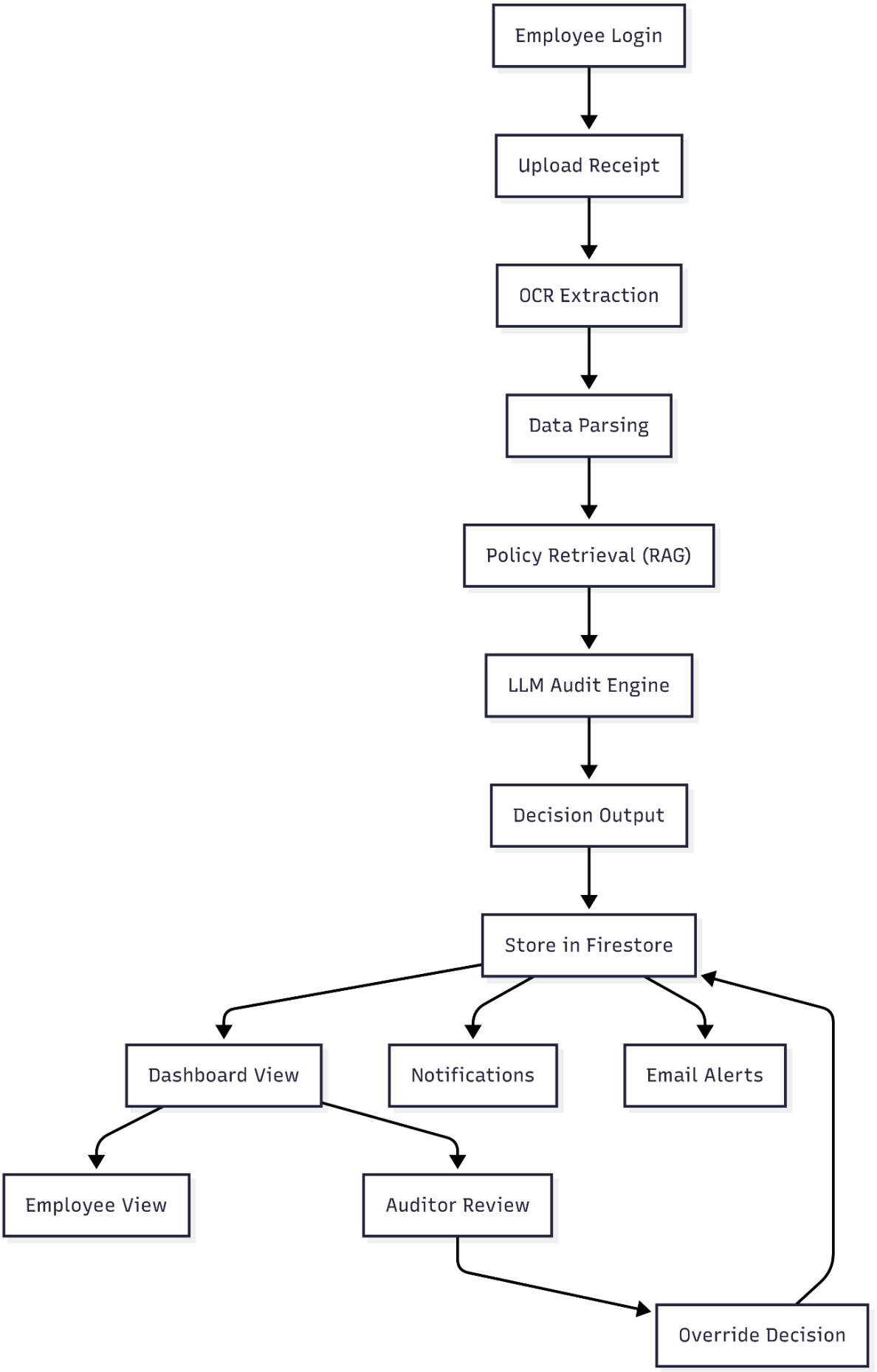
Corporate finance teams manually cross-reference employee expense receipts against complex Travel & Expense policies. This process is slow, inconsistent, and does not scale. Spending limits vary by city and seniority, policy rules can be ambiguous, and high volumes of receipts lead to delays and increased operational costs.

The core challenge is not just extracting data from receipts, but understanding the receipt, the employee's intent, and the relevant policy rules together to produce a clear and explainable decision.

2. Use Case Diagram



3. System Flow Diagram


Employee Login
Upload Receipt
OCR Extraction
Data Parsing
Policy Retrieval (RAG)
LLM Audit Engine
Decision Output
Store in Firestore
Dashboard View
Notifications
Email Alerts
Employee View
Auditor Review
Override Decision

4. Solution Design

ExpenseAudit is designed as a three-layer pipeline:

Layer 1 — Receipt Ingestion and OCR

Employees upload receipts in JPG, PNG, or PDF format. The system converts files into images if required and uses OCR to extract raw text. Structured parsing identifies key fields such as merchant name, transaction date, total amount, and currency.

A basic image quality check flags blurry or unreadable uploads. The system also verifies date consistency between the receipt and the submitted claim.

Layer 2 — Policy Retrieval and AI Audit

The company's policy document is divided into smaller sections. When a claim is submitted, relevant sections are selected based on keyword matching with the receipt and business purpose.

These sections, along with extracted receipt data, are sent to an AI model acting as a finance auditor. The system returns:

- Decision (Approved, Flagged, or Rejected)
- Reason based on policy
- Risk Level (Low, Medium, High)
- Expense Category (Meals, Travel, Lodging, etc.)
- Compliance Score (0–100)

Duplicate detection is also performed using merchant, amount, and date comparisons.

Layer 3 — Dashboard and Analytics

All claims are stored in Firestore and displayed in a dashboard sorted by risk level. Each entry shows category and compliance score.

Users can view:

- Personal expense history
- Detailed audit reports with receipt and policy references
- System decisions and explanations

Finance auditors can override decisions with comments. Notifications and email alerts are triggered at each stage.

5. Tech Stack Choices

- FastAPI (Python)
  Used for backend development due to speed, simplicity, and built-in API documentation.
- Pytesseract and pdf2image
  Used for OCR and PDF processing without external API costs.
- Keyword-based Retrieval
  Simple and efficient method for selecting relevant policy sections.
- Groq API with LLaMA Model
  Used for fast and accurate AI-based auditing.
- Firebase Auth and Firestore
  Used for authentication and real-time database storage.
- EmailJS
  Used for sending email notifications without backend SMTP setup.
- React, TypeScript, Tailwind CSS, Recharts
  Used for building a responsive frontend and analytics dashboard.

6. Future Improvements

- Semantic search using embeddings
- Multi-page PDF OCR support
- Confidence scoring for OCR results
- Role-based access control
- Cloud storage for receipts
- Spend leakage analytics

7. Summary

ExpenseAudit is an AI-powered expense auditing system that automates receipt processing, policy validation, and decision-making. It produces fast, consistent, and explainable results while reducing manual workload.

The system is simple, scalable, and can be extended into a production-ready solution.